

【第5回】 配列とオブジェクト・JSONについて

1. 配列について

1-1. 配列とは？

配列とは、**複数の値を順番にまとめて管理する入れ物**のことで、配列を使用することにより、JavaScript内で類似した変数をたくさん定義することなく、一つの配列で複数の値を処理することができる。

例)

【配列を使用しない場合】

```
let score1 = 94;
```

```
let score2 = 87;
```

```
let score3 = 78;
```

【配列を使用した場合】

```
let score = [94, 87, 78];
```

1-2. 配列の定義

配列は、以下のように定義し、一つの変数に代入し使用することができる。

```
let 配列変数名 = [カンマ区切りの配列内の値];
```

空の配列を定義する際には、以下のようなになる。

```
let 配列変数名 = [];
```

また、Arrayコンストラクタを使用して以下のようにも定義できる。

```
let 配列変数名 = new Array(カンマ区切りの配列内の値);
```

1. 配列について

1-2. 配列の定義

例)

`let score = [94, 87, 78];` → 値が入った配列を定義

`let score = [];` → 空の配列を定義

`let score = new Array(94, 87, 78);` → 値が入った配列を定義

1-3. 配列の利用

配列は、順番を指定して値を取り出したり、配列の要素数を取得したりして、**JavaScript**内で実装に利用する。

【配列から値を取り出す場合】

配列変数名[取り出したい値のインデックス(順番)]

※ **配列のインデックスは0からの開始となる**

【配列の要素数を取得する場合】

配列変数名.length

例)

`let score = [94, 87, 78];`

`alert(score[1]);` → 配列の2つ目の要素の値を取得して、ダイアログ表示する

`alert(score.length);` → 配列の要素の個数を取得して、ダイアログ表示する

1. 配列について

1-3. 配列の利用

また、配列を利用するに当たり、配列内の値を追加・変更・削除することも可能であり、それぞれの方法は以下の通りである。

【配列への値の追加】

- ・ 配列の末尾に追加

配列変数名.push([追加する値]);

- ・ 配列の先頭に追加

配列変数名.unshift([追加する値]);

【配列の値の変更】

配列変数名[変更したい値のインデックス] = [変更後の値];

【配列の値の削除】

- ・ 配列の末尾を削除

配列変数名.pop();

- ・ 配列の先頭を削除

配列変数名.shift();

- ・ インデックスを指定した値の削除

delete 配列変数名[削除したい値のインデックス];

※ pop・shiftで値を削除した場合には、配列そのものの要素数は減るが、deleteでインデックスを指定して削除した場合には要素数は変わらず、削除したインデックスの値がundefinedとなる

1. 配列について

1-3. 配列の利用

例)

let score = [94, 87, 78];

score.push(81); → 配列の末尾に「**81**」を追加

score.unshift(98); → 配列の先頭に「**98**」を追加

score[2] = 85; → 配列の3番目の要素の値を**85**に変更

score.pop(); → 配列の最後の要素を削除

score.shift(); → 配列の最初の要素を削除

delete score[1]; → 配列の2番目の要素の値を削除

1. 配列について

【例題1】

Javascript-sample-q1.html・**script-q1.js**・**style.css**をコピーし、**HTML**要素の読込完了後に以下を行う**JavaScript**を実装しなさい。

- ① **【好きな果物】**のプルダウンリストの中身を**script-q1.js**に定義している配列を使用して作成する。
- ② **【都道府県】**のプルダウンリストを**script-q1.js**に定義している配列を使用して作成する。
ただし、**1**行目には**value = 0**で空白を追加する。
- ③ **【性別】**の項目に**script-q1.js**に定義している配列を使用して、**input[type = 'radio']**の要素を動的に作成する。

2. 連想配列について

2-1. 連想配列とは？

連想配列とは通常の配列と異なり、配列内の値をキー名(値に対する項目名)と値で管理したもので、厳密にいうとJavaScriptでは他言語でいう連想配列という型は存在せず、連想配列を定義した際にはObject(オブジェクト)として扱われる。

【通常の配列との違い】

- ・ 通常の配列 … 値をインデックス(順番)で管理する
- ・ 連想配列 … 値をキー名で管理する

2-2. 連想配列の定義

連想配列は、以下のように定義し、一つの変数に代入し使用することができる。

let 連想配列変数名 = {[キー名]: [値], [キー名]: [値], ...};

空の配列を定義する際には、以下のようなになる。

let 連想配列変数名 = {};

例)

```
let user = {  
  name: '田中'  
  ,age: 36  
  ,job: 'プログラマー'  
};
```

2. 連想配列について

2-3. 連想配列の利用

連想配列は、キー名を指定して値を取り出したり、値を変更したりして、**JavaScript**内で実装に利用する。キー名を指定した連想配列利用時の記述方法は以下の2種類がある。

【ドット記述法】

連想配列変数名.キー名

【ブラケット記述法】

連想配列変数名['キー名']

例)

```
let user = {  
  name : '田中'  
  ,age : 36  
  ,job : 'プログラマー'  
};
```

`alert(user.name);` → 連想配列`user`内のキー名`name`の値をダイアログ表示

`alert(user['name']);` → 連想配列`user`内のキー名`name`の値をダイアログ表示

`user.age = 30;` → 連想配列`user`内のキー名`age`の値を`30`に変更する

※ 連想配列のキー名の指定を文字列変数を使用して行う場合には、ドット記述法は使用できないため、ブラケット記述法を利用する

2. 連想配列について

2-3. 連想配列の利用

また、連想配列でも通常の配列と同様に、連想配列内の値を追加・削除することも可能である。連想配列に対して新たなキー名を指定し、値を代入すると追加される。通常の配列の`delete`を使用すると、指定したキー名の値を削除することができる。

例)

```
let user = {  
  name : '田中'  
  ,age : 36  
  ,job : 'プログラマー'  
};
```

`user.email = 'text@it-blade.co.jp';` → 連想配列`user`にキー名`email`の値を追加する

`delete user.email;` → 連想配列`user`からキー名`email`の値を削除する

連想配列内の要素で反復処理を行う際には、通常の配列とは記述が異なり、以下のように記述する。

```
for([連想配列のキー名用の変数定義] in [連想配列名])  
{  
  [繰り返し行う処理]  
}
```

2. 連想配列について

2-3. 連想配列の利用

```
例)
  for(let key in user)
  {
    alert(user[key]);
  }
```

2-4. 連想配列の応用

連想配列は**Object**(オブジェクト)であるため、**JavaScript**でオブジェクトを扱う際の以下の関数を利用することができる。

【連想配列のキー名の一覧を取得する】

Object.keys(連想配列名);
→ キー名のみが配列で返ってくる

【連想配列の値の一覧を取得する】

Object.values(連想配列名);
→ 値のみが配列で返ってくる

【連想配列の値の一覧を取得する】

Object.entries(連想配列名);
→ キー名と値の組み合わせの配列が配列に格納されて返ってくる

2. 連想配列について

2-4. 連想配列の応用

例)

```
let user = {  
  name : '田中'  
  ,age : 36  
  ,job : 'プログラマー'  
};
```

let keys = Object.keys(user); → ['name', 'age', 'job']が返ってくる

let vals = Object.values(user); → ['田中', 36, 'プログラマー']が返ってくる

let objs = Object.entries(user); → [['name', '田中'], ['age', 36], ['job', 'プログラマー']]が返ってくる

また、連想配列は連想配列単体だけではなく、以下のように通常の配列と組み合わせて使用することも可能である。

【連想配列と配列の組み合わせ】

```
let 変数名 = [  
  連想配列  
  ,連想配列  
];
```

```
let 連想配列名 = {  
  キー名 : 配列  
  ,キー名 : 配列  
};
```

2. 連想配列について

2-4. 連想配列の応用

例)

```
let users = [  
  {name: '田中', age: 36, job: 'プログラマー'}  
  , {name: '佐藤', age: 42, job: 'システムエンジニア'}  
  , {name: '鈴木', age: 46, job: 'プロジェクトマネージャー'}  
];
```

```
let users2 = {  
  names: ['田中', '佐藤', '鈴木']  
  , ages: [36, 42, 46]  
  , jobs: ['プログラマー', 'システムエンジニア', 'プロジェクトマネージャー']  
};
```

`alert(users[1].job);` → 「システムエンジニア」がダイアログ表示される

`alert(users2.ages[1]);` → 「42」がダイアログ表示される

2. 連想配列について

【例題2】

【例題1】で作成したファイルに以下を行う**JavaScript**を追加で実装しなさい。

- ① **[入力結果の追加]**ボタンをクリックした際に、**HTML**上のフォームに入力された結果を連想配列に取得し、フォーム下の**table**要素に行を追加する。

3. JSONについて

3-1. JSONとは?

JSONとは、JavaScriptの連想配列(**Object**)から派生して作られたサーバーとデータをやり取りするための文字列フォーマットのことである。

見た目は似ているが、JSONはあくまでも文字列なので、以下のような違いがある。

項目	連想配列	JSON
データ型	JavaScript内の値に依存	全て文字列
関数	JavaScriptでのObjectで使用可能な関数が利用可能	文字列なので、関数の利用は不可
キー名	クォートは不要	クォートは必須
通信での利用	サーバーとの通信では利用不可	サーバーとの通信での利用可

例)

【連想配列】

```
let user = {name : '田中', age : 36, job : 'プログラマー'};
```

【JSON】

```
let user_json = {'name' : '田中', 'age' : 36, 'job' : 'プログラマー'};
```

3. JSONについて

3-2. JSONの利用

JSONを利用するには、連想配列でのドット記述法やブラケット記述法などの記述方法で値を使用することができるが、データ型が文字列のため、そのままではデータ型を要求されるような処理(数値の計算など)には使用することができない。

JSONをそれぞれの値に応じてデータ型を持ったObjectに変換することで、データ型を要求されるような処理でも利用可能となる。

また、JSONはデータ通信に利用することが可能なため、ObjectをJSONに変換することで、WEB APIのリクエストパラメータ等のサーバーとの通信のパラメータとして利用することができる。

【ObjectとJSONの相互変換】

- Object → JSON

JSON.stringify(Object);

- JSON → Object

JSON.parse(JSON文字列);

例)

【Object → JSON】

```
let user = {name : '田中', age : 36, job : 'プログラマー'};
```

let json = JSON.stringify(user); → jsonに変換されたJSON文字列が格納される

【JSON → Object】

```
let user_json = {'name' : '田中', 'age' : 36, 'job' : 'プログラマー'};
```

let obj = JSON.parse(user_json); → objに変換されたObjectが格納される

3. JSONについて

【例題3】

【例題2】で作成したファイルに以下を行う**JavaScript**を追加で実装しなさい。

- ① **[入力結果の表示]**ボタンをクリックした際に、**HTML**上のフォームに入力された結果を**JSON**文字列としてダイアログに表示する。